

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

FACULTY : Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I GUZHALINITHA.K – 192425232 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:10

Total Marks:50

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

6. Perceptron and Multilayer Perceptron (MLP)

- a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.
- b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

7. Forward Propagation and Backpropagation Algorithm

- a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.
- b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

- a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.
- b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. Learning Algorithms

Aim

Implement Linear Regression using Gradient Descent and observe loss reduction.

Algorithm

1. Initialize weight and bias.
2. Calculate predicted values.
3. Compute Mean Squared Error (MSE).
4. Update weight and bias using Gradient Descent.
5. Repeat until loss decreases.

Result Analysis

- Initially, the loss is high because parameters are randomly initialized.

- With each iteration, the loss decreases.
 - The learning curve shows smooth convergence.
 - The final model accurately predicts the output values.
-

2. Maximum Likelihood Estimation (MLE)

Aim

Estimate the mean and variance of data generated from a normal distribution.

Procedure

1. Generate synthetic data.
2. Compute mean using MLE.
3. Compute variance using MLE.
4. Compare estimated values with actual values.

Result Analysis

- Estimated mean is close to the true mean.
 - Estimated variance is close to the actual variance.
 - Small differences occur due to random sampling.
 - MLE provides reliable parameter estimates with sufficient data.
-

3. Building Machine Learning Algorithms

Aim

Build a complete classification model.

Steps

1. Load the dataset.
2. Preprocess data.
3. Split into training and testing sets.
4. Train a classifier.
5. Predict test data.
6. Evaluate using Accuracy and Confusion Matrix.

Result Analysis

- High accuracy indicates good classification.
 - Confusion matrix shows correct and incorrect predictions.
 - Data preprocessing improves performance.
 - The trained model successfully classifies unseen data.
-

4. Neural Networks

Aim

Design a neural network with one hidden layer.

Procedure

1. Create input layer.
2. Add one hidden layer.
3. Use an activation function.
4. Train using backpropagation.
5. Evaluate predictions.

Result Analysis

- Hidden neurons learn complex patterns.
 - Training loss decreases over epochs.
 - The network correctly learns non-linear relationships.
 - Prediction accuracy improves after training.
-

5. Artificial Neurons

Aim

Compare Sigmoid and ReLU activation functions.

Observation

Activation Function Output Characteristics

Sigmoid Output between 0 and 1

ReLU Negative values become 0 and positive values remain unchanged

Result Analysis

- Sigmoid is suitable for probability-based outputs.
 - ReLU trains faster.
 - ReLU reduces the vanishing gradient problem.
 - ReLU performs better in deep networks.
-

6(a). Perceptron

Aim

Test Perceptron on different datasets.

Result Analysis

- Performs well on linearly separable data.
- Fails to classify non-linearly separable data.
- Decision boundary is linear.
- Accuracy decreases for complex datasets.

6(b). Multilayer Perceptron (MLP)

Aim

Compare MLP with Perceptron.

Result Analysis

Perceptron	MLP
Single layer	Multiple layers
Solves linear problems	Solves linear and non-linear problems
Lower accuracy	Higher accuracy
No hidden layer	Hidden layer present

Conclusion

MLP performs better because hidden layers learn complex patterns.

7(a). Forward Propagation

Aim

Compute forward propagation.

Steps

1. Multiply inputs with weights.
2. Add bias.
3. Apply activation function.
4. Produce output.

Result Analysis

- Forward propagation computes the network output.
 - Code verification matches manual calculation.
 - Output depends on weights and activation function.
 - Correct implementation produces accurate predictions.
-

7(b). Backpropagation

Aim

Update weights after one iteration.

Steps

1. Calculate prediction error.
2. Compute gradients.
3. Update weights.

4. Repeat for all layers.

Result Analysis

- Weight values change after one iteration.
 - Error decreases after updating weights.
 - Repeated updates improve learning.
 - Backpropagation minimizes prediction error.
-

8(a). Gradient Descent

Aim

Study the effect of learning rate.

Result Analysis

- Small learning rate:
 - Slow convergence.
 - Stable training.
- Moderate learning rate:
 - Faster convergence.
 - Better performance.
- Large learning rate:
 - Overshoots the minimum.
 - May not converge.

Conclusion

Choosing an appropriate learning rate is essential for efficient training.

8(b). Stochastic Gradient Descent (SGD)

Aim

Compare SGD with Batch Gradient Descent.

Comparison

Batch GD	SGD
Uses complete dataset	Uses one sample at a time
Slower	Faster
Smooth convergence	Noisy convergence
High computation	Lower computation

Result Analysis

- SGD updates parameters more frequently.

- Training is faster.
 - Loss fluctuates during learning.
 - Both methods converge to good solutions.
-

9. Mini-Batch Gradient Descent

Aim

Analyze the effect of batch size.

Result Analysis

- Small batch size updates weights frequently.
 - Large batch size gives smoother convergence.
 - Medium batch size provides the best balance.
 - Mini-Batch Gradient Descent is faster and more stable than Batch Gradient Descent.
-

10. Curse of Dimensionality

Aim

Analyze the effect of increasing dimensions.

Observation

Low Dimensions	High Dimensions
Points are close	Points become sparse
Easier learning	Difficult learning
Better accuracy	Accuracy may decrease
Lower computation	Higher computation

Result Analysis

- Distance between data points increases with dimensionality.
 - Data becomes sparse.
 - Model performance decreases.
 - Feature selection and dimensionality reduction help improve performance.
-

Overall Conclusion

The experiments demonstrated the implementation of key deep learning concepts, including learning algorithms, MLE, neural networks, perceptrons, forward and backpropagation, Gradient Descent, SGD, Mini-Batch Gradient Descent, and the Curse of Dimensionality. The results showed that appropriate model design, activation functions, optimization techniques, and dimensionality reduction significantly improve learning efficiency, convergence, and prediction accuracy.